

Policy gradient Methods

Master Degree in Control Systems Engineering 2024-2025
University of Padova

Teachers: Prof. Ruggero Carli & Prof. Gian Antonio Susto

Second part of the Ttday's lecture

- Introduction to the problem
- Policy gradient theorem
- REINFORCE algorithm

Policy-Based Reinforcement Learning

- In the last lectures we approximated the value or action-value function using parameter w

$$\begin{aligned}\hat{v}(s, w) &\approx v_{\pi}(s) \\ \hat{q}(s, a, w) &\approx q_{\pi}(s, a)\end{aligned}$$

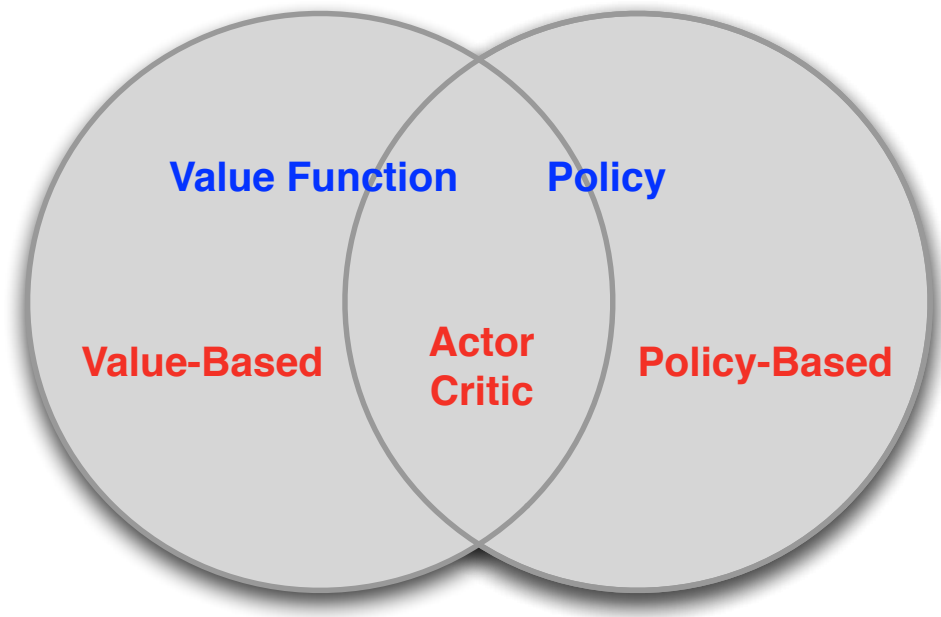
- A policy was generated directly from the value function (e.g., using ϵ -greedy)
- In this lecture we will directly parametrize the **policy**

$$\pi(a|s, \theta) = \Pr\{A_t = a \mid S_t = s, \theta_t = \theta\}$$

- We will focus again on **model-free** reinforcement learning

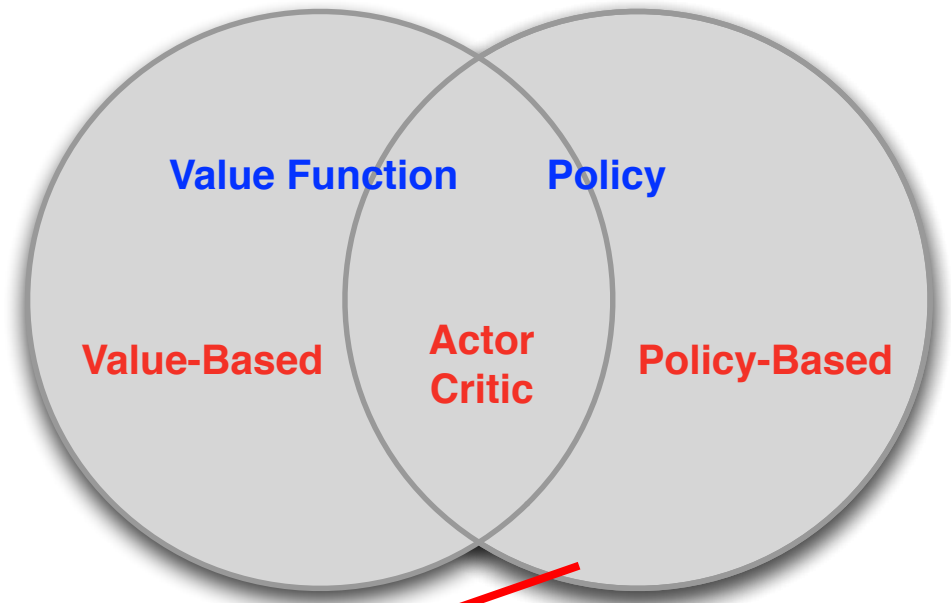
Value-Based and Policy-Based RL

- Value Based
 - Learnt Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learnt Policy
- Actor-Critic
 - Learnt Value Function
 - Learnt Policy



Value-Based and Policy-Based RL

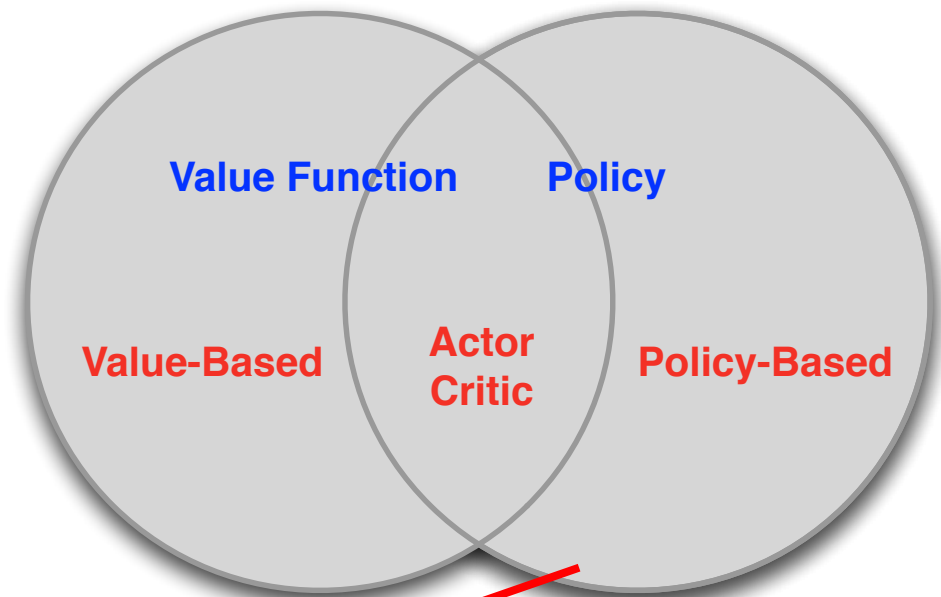
- Value Based
 - Learnt Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learnt Policy
- Actor-Critic
 - Learnt Value Function
 - Learnt Policy



$J(\theta)$: performance measure

Value-Based and Policy-Based RL

- Value Based
 - Learnt Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learnt Policy
- Actor-Critic
 - Learnt Value Function
 - Learnt Policy

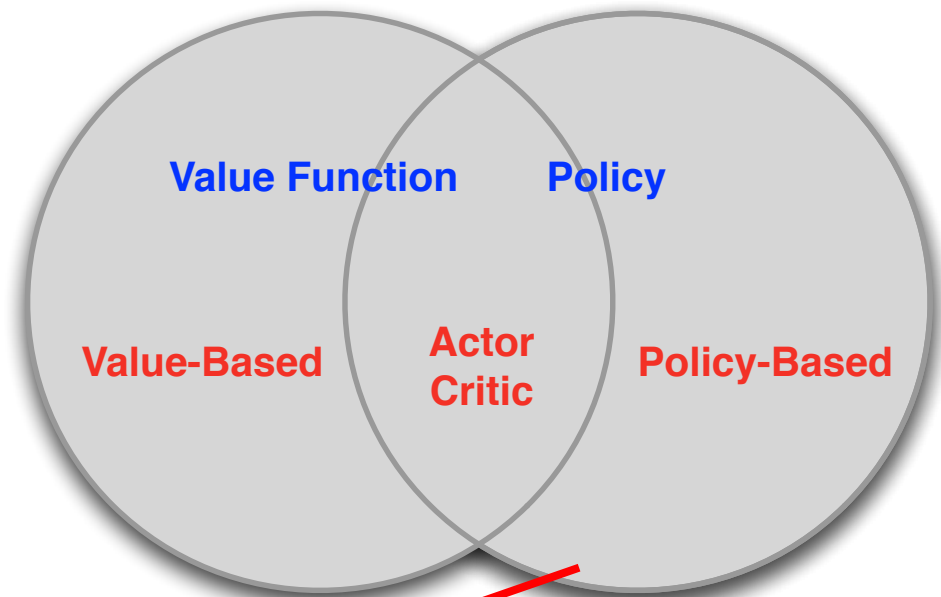


$J(\theta)$: performance measure

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla J(\theta_t)} \quad \text{gradient ascent}$$

Value-Based and Policy-Based RL

- Value Based
 - Learnt Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learnt Policy
- Actor-Critic
 - Learnt Value Function
 - Learnt Policy



$J(\theta)$: performance measure

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla J(\theta_t)} \quad \text{gradient ascent}$$

$\widehat{\nabla J(\theta_t)}$ is a stochastic estimate whose expectation approximates the gradient of the performance measure with respect to its argument

Advantages of Policy-Based RL

Advantages:

- Better convergence properties
- Effective in high-dimensional or continuous action spaces
- Can learn stochastic policies

Disadvantages:

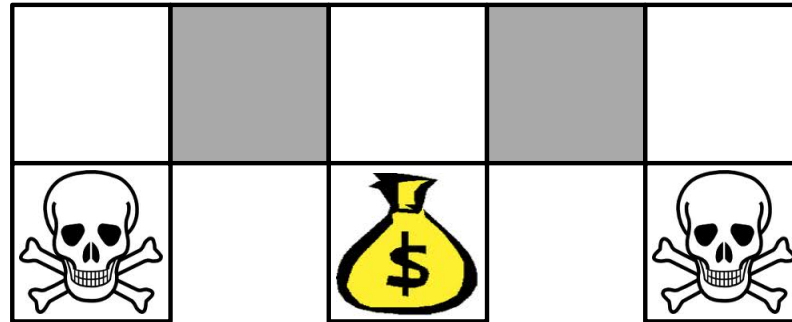
- Typically converge to a local rather than global optimum
- Evaluating a policy is typically inefficient and high variance

Example : Rock-Paper-Scissors



- Two-player game of rock-paper-scissors
 - Scissors beats paper
 - Rock beats scissors
 - Paper beats rock
- Consider policies for *iterated* rock-paper-scissors
 - A deterministic policy is easily exploited
 - A uniform random policy is optimal (i.e. Nash equilibrium)

Example : Aliased Gridworld



- The agent cannot differentiate the grey states
- Consider features of the following form (for all N, E, S, W)

$$\phi(s, a) = \mathbf{1}(\text{wall to N, } a = \text{move E})$$

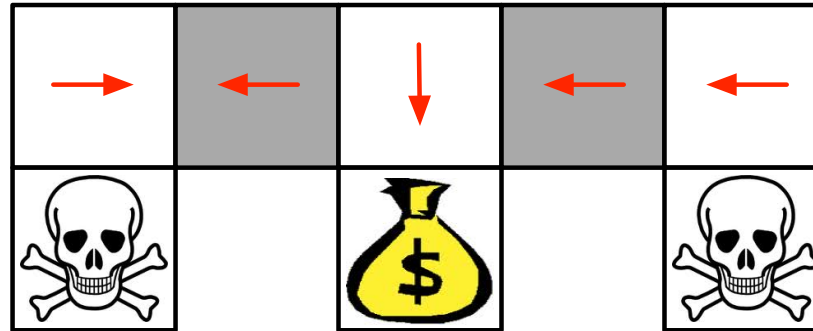
- Compare value-based RL, using an approximate value function

$$Q_{\theta}(s, a) = f(\phi(s, a), \theta)$$

- To policy-based RL, using a parametrised policy

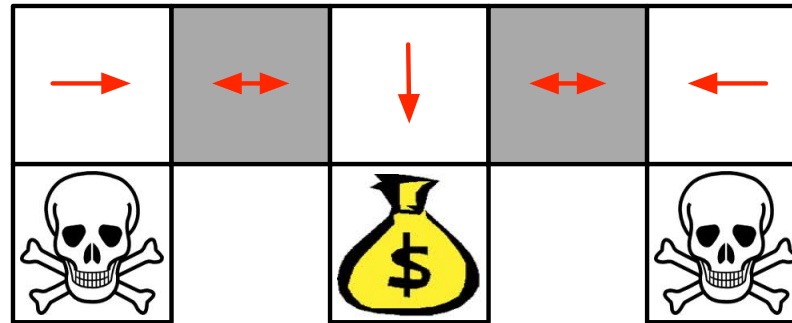
$$\pi_{\theta}(s, a) = g(\phi(s, a), \theta)$$

Example : Aliased Gridworld



- Under aliasing, an optimal **deterministic** policy will either
 - move W in both grey states (shown by red arrows)
 - move E in both grey states
- Either way, it can get stuck and *never* reach the money
- Value-based RL learns a near-deterministic policy
 - e.g. greedy or ϵ -greedy
- So it will traverse the corridor for a long time

Example : Aliased Gridworld



- An optimal **stochastic** policy will randomly move E or W in grey states

$$\pi_{\theta}(\text{wall to N and S, move E}) = 0.5$$

$$\pi_{\theta}(\text{wall to N and S, move W}) = 0.5$$

- It will reach the goal state in a few steps with high probability
- Policy-based RL can learn the optimal stochastic policy

A typical Policy parametrization

- $\pi(a|s, \theta)$ is differentiable with the respect its parameters
- $\nabla_{\theta}\pi(a|s, \theta)$ exists and is always finite
- to ensure exploration we generally require that the policy never becomes deterministic

A typical Policy parametrization

- $\pi(a|s, \boldsymbol{\theta})$ is differentiable with the respect its parameters
- $\nabla_{\boldsymbol{\theta}}\pi(a|s, \boldsymbol{\theta})$ exists and is always finite
- to ensure exploration we generally require that the policy never becomes deterministic

Exponential Softmax Distribution

$$\pi(a|s, \boldsymbol{\theta}) = \frac{\exp(h(s, a, \boldsymbol{\theta}))}{\sum_b \exp(h(s, b, \boldsymbol{\theta}))}$$

$h(s, a, \theta)$ parametrized numerical preferences

Policy Objective function

We consider the episodic case

We define the *performance measure* as the *value of the start state of the episode* (we assume that every episode starts in some particular (non-random) state s_0)

$$J(\theta) \doteq v_{\pi_\theta}(s_0)$$

v_{π_θ} is the *true value function* for π_θ , the policy determined by θ

Policy gradient

- Let $J(\theta)$ be any policy objective function
- Policy gradient algorithms search for a *local* maximum in $J(\theta)$ by ascending the gradient of the policy, w.r.t. parameters θ

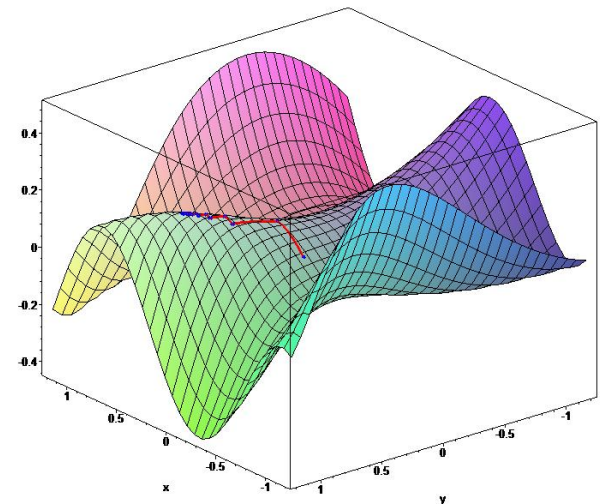
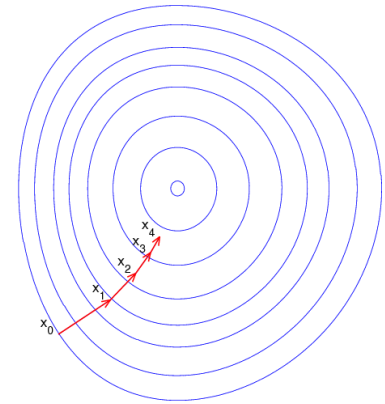
$$\Delta\theta = \alpha \nabla_{\theta} J(\theta)$$

- Where $\nabla_{\theta} J(\theta)$ is the **policy gradient**

$$\nabla_{\theta} J(\theta) = \begin{pmatrix} \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{pmatrix}$$

- and α is a step-size parameter

$$\theta_{t+1} = \theta_t + \alpha \Delta\theta$$



Computing gradients by finite differences

- To evaluate policy gradient of $\pi_{\theta}(s, a)$
- For each dimension $k \in [1, n]$
 - Estimate k th partial derivative of objective function w.r.t. θ
 - By perturbing θ by small amount ϵ in k th dimension

$$\frac{\partial J(\theta)}{\partial \theta_k} \approx \frac{J(\theta + \epsilon u_k) - J(\theta)}{\epsilon}$$

where u_k is unit vector with 1 in k th component, 0 elsewhere

- Uses n evaluations to compute policy gradient in n dimensions
- Simple, noisy, inefficient - but sometimes effective
- Works for arbitrary policies, even if policy is not differentiable

The policy gradient theorem

We would like to compute the gradient analytically...

The policy gradient theorem

We would like to compute the gradient analytically...

With function approximation, it may seem challenging to change the policy parameter in a way that ensures improvement.

The policy gradient theorem

We would like to compute the gradient analytically...

With function approximation, it may seem challenging to change the policy parameter in a way that ensures improvement.

Performance depends on :

- *action selections*
- *distribution of states* (in which those selections are made)

The policy gradient theorem

We would like to compute the gradient analytically...

With function approximation, it may seem challenging to change the policy parameter in a way that ensures improvement.

Performance depends on :

- *action selections*
- *distribution of states* (in which those selections are made)

Both of them are affected by the policy parameter!

The policy gradient theorem

We would like to compute the gradient analytically...

With function approximation, it may seem challenging to change the policy parameter in a way that ensures improvement.

Performance depends on :

- *action selections*
- *distribution of states* (in which those selections are made)

OBSERVATIONS:

1. Given a state, the *effect of the policy parameter on the actions*, and thus on reward, can be computed in a *relatively straightforward* way from knowledge of the parameterization...
2. ...but the *effect of the policy on the state distribution* is a function of the environment and is *typically unknown*.

The policy gradient theorem

We would like to compute the gradient analytically...

With function approximation, it may seem challenging to change the policy parameter in a way that ensures improvement.

Performance depends on :

- *action selections*
- *distribution of states* (in which those selections are made)

How can we estimate the performance gradient with respect to the policy parameter when the gradient depends on the unknown effect of policy changes on the state distribution?

The policy gradient theorem

We would like to compute the gradient analytically...

With function approximation, it may seem challenging to change the policy parameter in a way that ensures improvement.

Performance depends on :

- *action selections*
- *distribution of states* (in which those selections are made)

How can we estimate the performance gradient with respect to the policy parameter when the gradient depends on the unknown effect of policy changes on the state distribution?



Policy Gradient Theorem

The policy gradient theorem

Policy gradient theorem provides an *analytic expression for the gradient* of performance with respect to the policy parameter that does *not involve the derivative of the state distribution*.

The policy gradient theorem

Policy gradient theorem provides an *analytic expression for the gradient* of performance with respect to the policy parameter that does *not involve the derivative of the state distribution*.

Policy Gradient Theorem

The policy gradient theorem for the episodic case establishes that

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_{\pi}(s, a) \nabla_{\boldsymbol{\theta}} \pi(a|s, \boldsymbol{\theta})$$

- The gradients are column vectors of partial derivatives with respect to the components of $\boldsymbol{\theta}$
- The symbol $\propto$ here means “proportional to”

The policy gradient theorem

Policy gradient theorem provides an *analytic expression for the gradient* of performance with respect to the policy parameter that does *not involve the derivative of the state distribution*.

Policy Gradient Theorem

The policy gradient theorem for the episodic case establishes that

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla_{\boldsymbol{\theta}} \pi(a|s, \boldsymbol{\theta})$$

- In the episodic case, the constant of proportionality is the average length of an episode, and in the continuing case it is 1, so that the relationship is actually an equality*

Toward REINFORCE : Monte Carlo Policy Gradient

Stochastic gradient ascent

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \widehat{\nabla J(\boldsymbol{\theta}_t)}$$

Toward REINFORCE : Monte Carlo Policy Gradient

Stochastic gradient ascent

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \widehat{\nabla J(\boldsymbol{\theta}_t)}$$

We need a way *to obtain samples such that the expectation of the sample gradient is proportional to the actual gradient of the performance measure as a function of the parameter*

Toward REINFORCE : Monte Carlo Policy Gradient

Stochastic gradient ascent

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla J(\theta_t)}$$

We need a way *to obtain samples such that the expectation of the sample gradient is proportional to the actual gradient of the performance measure as a function of the parameter*



The sample gradients need only be proportional to the gradient because any constant of proportionality can be absorbed into the step size α , which is otherwise arbitrary.

Toward REINFORCE : Monte Carlo Policy Gradient

The policy gradient theorem gives an exact expression proportional to the gradient; all that is needed is some way of sampling whose expectation equals or approximates this expression.

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_{\pi}(s, a) \nabla_{\boldsymbol{\theta}} \pi(a|s, \boldsymbol{\theta}),$$

Toward REINFORCE : Monte Carlo Policy Gradient

The policy gradient theorem gives an exact expression proportional to the gradient; all that is needed is some way of sampling whose expectation equals or approximates this expression.

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla_{\boldsymbol{\theta}} \pi(a|s, \boldsymbol{\theta}),$$

Observation : the right-hand side of the policy gradient theorem is a sum over states weighted by how often the states occur under the target policy π

Toward REINFORCE : Monte Carlo Policy Gradient

The policy gradient theorem gives an exact expression proportional to the gradient; all that is needed is some way of sampling whose expectation equals or approximates this expression.

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla_{\boldsymbol{\theta}} \pi(a|s, \boldsymbol{\theta}),$$

Observation : the right-hand side of the policy gradient theorem is a sum over states weighted by how often the states occur under the target policy π

if π is followed, then states will be encountered in these proportions

Toward REINFORCE : Monte Carlo Policy Gradient

The policy gradient theorem gives an exact expression proportional to the gradient; all that is needed is some way of sampling whose expectation equals or approximates this expression.

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla_{\boldsymbol{\theta}} \pi(a|s, \boldsymbol{\theta}),$$

Observation : the right-hand side of the policy gradient theorem is a sum over states weighted by how often the states occur under the target policy π

if π is followed, then states will be encountered in these proportions


$$= \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \nabla_{\boldsymbol{\theta}} \pi(a|S_t, \boldsymbol{\theta}) \right].$$

Toward REINFORCE : Monte Carlo Policy Gradient

$$\nabla J(\boldsymbol{\theta}) \propto \mathbb{E}_{\pi} \left[\sum_a q_{\pi}(S_t, a) \nabla_{\boldsymbol{\theta}} \pi(a|S_t, \boldsymbol{\theta}) \right]$$

Now we would like to carry it further and handle the action in the same way (replacing a with the sample action A_t).

The remaining part of the expectation above is a sum over actions; if only each term were weighted by the probability of selecting the actions, that is, according to $\pi(a|S_t, \theta)$, then the replacement could be done.

Toward REINFORCE : Monte Carlo Policy Gradient

$$\nabla J(\boldsymbol{\theta}) \propto \mathbb{E}_{\pi} \left[\sum_a q_{\pi}(S_t, a) \nabla_{\boldsymbol{\theta}} \pi(a|S_t, \boldsymbol{\theta}) \right]$$


Now we would like to carry it further and handle the action in the same way (replacing a with the sample action A_t).

The remaining part of the expectation above is a sum over actions; if only each term were weighted by the probability of selecting the actions, that is, according to $\pi(a|S_t, \theta)$, then the replacement could be done.

$$= \mathbb{E}_{\pi} \left[\sum_a \pi(a|S_t, \boldsymbol{\theta}) q_{\pi}(S_t, a) \frac{\nabla_{\boldsymbol{\theta}} \pi(a|S_t, \boldsymbol{\theta})}{\pi(a|S_t, \boldsymbol{\theta})} \right]$$

Toward REINFORCE : Monte Carlo Policy Gradient

$$\nabla J(\boldsymbol{\theta}) \propto \mathbb{E}_{\pi} \left[\sum_a q_{\pi}(S_t, a) \nabla_{\boldsymbol{\theta}} \pi(a|S_t, \boldsymbol{\theta}) \right]$$


Now we would like to carry it further and handle the action in the same way (replacing a with the sample action A_t).

The remaining part of the expectation above is a sum over actions; if only each term were weighted by the probability of selecting the actions, that is, according to $\pi(a|S_t, \theta)$, then the replacement could be done.

$$\begin{aligned} &= \mathbb{E}_{\pi} \left[\sum_a \pi(a|S_t, \boldsymbol{\theta}) q_{\pi}(S_t, a) \frac{\nabla_{\boldsymbol{\theta}} \pi(a|S_t, \boldsymbol{\theta})}{\pi(a|S_t, \boldsymbol{\theta})} \right] \\ &= \mathbb{E}_{\pi} \left[q_{\pi}(S_t, A_t) \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t|S_t, \boldsymbol{\theta})}{\pi(A_t|S_t, \boldsymbol{\theta})} \right] \end{aligned}$$

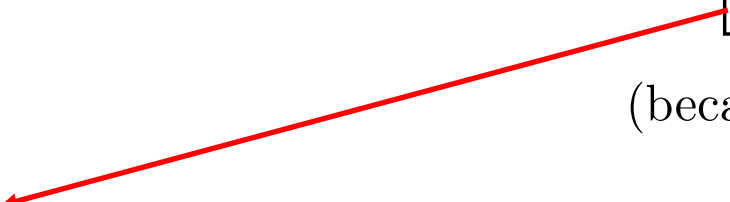
(replacing a by the sample $A_t \sim \pi$)

Toward REINFORCE : Monte Carlo Policy Gradient

$$\nabla J(\boldsymbol{\theta}) \propto \mathbb{E}_{\pi} \left[q_{\pi}(S_t, A_t) \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta})}{\pi(A_t | S_t, \boldsymbol{\theta})} \right]$$

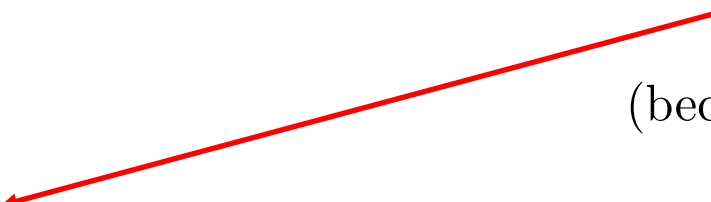
$$= \mathbb{E}_{\pi} \left[G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta})}{\pi(A_t | S_t, \boldsymbol{\theta})} \right]$$

(because $\mathbb{E}_{\pi}[G_t | S_t, A_t] = q_{\pi}(S_t, A_t)$)



G_t is the return!

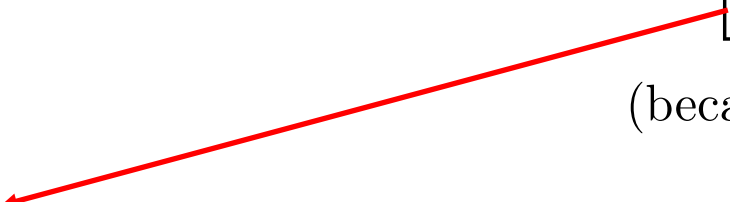
Toward REINFORCE : Monte Carlo Policy Gradient

$$\begin{aligned}\nabla J(\boldsymbol{\theta}) &\propto \mathbb{E}_{\pi} \left[q_{\pi}(S_t, A_t) \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta})}{\pi(A_t | S_t, \boldsymbol{\theta})} \right] \\ &= \mathbb{E}_{\pi} \left[G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta})}{\pi(A_t | S_t, \boldsymbol{\theta})} \right] \\ &\quad (\text{because } \mathbb{E}_{\pi}[G_t | S_t, A_t] = q_{\pi}(S_t, A_t))\end{aligned}$$


G_t is the return!

This expression in the brackets is exactly what is needed, a quantity that can be sampled on each time step whose expectation is equal to the gradient

Toward REINFORCE : Monte Carlo Policy Gradient

$$\begin{aligned}\nabla J(\boldsymbol{\theta}) &\propto \mathbb{E}_{\pi} \left[q_{\pi}(S_t, A_t) \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta})}{\pi(A_t | S_t, \boldsymbol{\theta})} \right] \\ &= \mathbb{E}_{\pi} \left[G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta})}{\pi(A_t | S_t, \boldsymbol{\theta})} \right] \\ &\quad (\text{because } \mathbb{E}_{\pi}[G_t | S_t, A_t] = q_{\pi}(S_t, A_t))\end{aligned}$$


G_t is the return!

This expression in the brackets is exactly what is needed, a quantity that can be sampled on each time step whose expectation is equal to the gradient

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \widehat{\nabla J(\boldsymbol{\theta}_t)} \quad \longrightarrow \quad \boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

REINFORCE

REINFORCE : Monte Carlo Policy Gradient

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

REINFORCE : Monte Carlo Policy Gradient

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

The update increases the parameter vector in this direction *(i) proportional to the return*, and *(ii) inversely proportional to the action probability*.

REINFORCE : Monte Carlo Policy Gradient

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

The update increases the parameter vector in this direction *(i) proportional to the return*, and *(ii) inversely proportional to the action probability*.

- (i) makes sense because it causes the parameter to move most in the directions that favor actions that yield the highest return.

REINFORCE : Monte Carlo Policy Gradient

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}$$

The update increases the parameter vector in this direction *(i) proportional to the return*, and *(ii) inversely proportional to the action probability*.

- (i) makes sense because it causes the parameter to move most in the directions that favor actions that yield the highest return.
- (ii) makes sense because otherwise actions that are selected frequently are at an advantage (the updates will be more often in their direction) and might win out even if they do not yield the highest return.

REINFORCE : Monte Carlo Policy Gradient

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)} \doteq \boldsymbol{\theta}_t + \alpha G_t \nabla_{\boldsymbol{\theta}} \ln \pi(A_t | S_t, \boldsymbol{\theta}_t).$$

REINFORCE : Monte Carlo Policy Gradient

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)} \doteq \boldsymbol{\theta}_t + \alpha G_t \nabla_{\boldsymbol{\theta}} \ln \pi(A_t | S_t, \boldsymbol{\theta}_t).$$



Score function

REINFORCE : Monte Carlo Policy Gradient

REINFORCE uses the complete return from time t , which includes all future rewards up until the end of the episode.



REINFORCE is a Monte Carlo algorithm

REINFORCE : Monte Carlo Policy Gradient

A Monte-Carlo Policy-Gradient Method (episodic) for estimating $\pi_\theta \approx \pi_*$

Input : a differentiable policy parametrization $\pi(a|s, \theta)$

Algorithm parameter : step-size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ arbitrarily (e.g. $\theta = 0$)

Repeat (for each episode) :

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot | \cdot, \theta)$:

 Loop for each step of the episode $t = 0, \dots, T - 1$

$G \leftarrow$ return from step t (G_t)

$\theta \leftarrow \theta + \alpha G \nabla_\theta \ln \pi(A_t | S_t, \theta)$

REINFORCE : Monte Carlo Policy Gradient

REINFORCE uses the complete return from time t , which includes all future rewards up until the end of the episode.



REINFORCE is a Monte Carlo algorithm

FINAL OBSERVATIONS :

1. As a stochastic gradient method, REINFORCE has *good theoretical convergence properties* ; convergence to a local optimum under standard stochastic approximation conditions for decreasing α

REINFORCE : Monte Carlo Policy Gradient

REINFORCE uses the complete return from time t , which includes all future rewards up until the end of the episode.



REINFORCE is a Monte Carlo algorithm

FINAL OBSERVATIONS :

1. As a stochastic gradient method, REINFORCE has *good theoretical convergence properties* ; convergence to a local optimum under standard stochastic approximation conditions for decreasing α
2. as a Monte Carlo method REINFORCE may be of *high variance* and thus produce *slow learning*.

Material

The material illustrated in this first part can be found in the first three paragraphs of Chapter 13.